

STATS 7022 - Data Science Assignment 3: Question 1

Dongju Ma

Trimester 1, 2025

(a) Read in the data

```
pacman::p_load(tidyverse, tidymodels, kernlab)
a3 <- read_rds('./data/a3_data.rds')
a3
```

```
## # A tibble: 6,000 x 7
##   Y      X1      X2      X3      X4 C1      C2
##   <fct> <chr>    <dbl>    <dbl>    <dbl> <fct> <fct>
## 1 B      2.09993193230127 -0.0543 -2.71    0.00119 w      V
## 2 B      1.2277356279852 -1.59   -1.37   -0.167  z      U
## 3 B      0.734113591160953 -1.44    0.0747  0.144  w      U
## 4 B      1.37285625816337 -0.467  -1.04    0.122  w      T
## 5 A      3.18496377283462  0.591  -0.219   0.346  y      U
## 6 B      1.14530421679377 -0.556  -0.847   0.0811 x      V
## 7 A      2.01235934125618 -1.98   -0.727  -1.99   w      T
## 8 A     -0.109220689724606  1.19   -1.37    0.456  y      T
## 9 B      2.44469893797972  0.0826  1.60   -1.29   y      V
## 10 A     0.76684685834349  0.587  -1.38   -0.154  z      V
## # i 5,990 more rows
```

There are 6,000 observations in the original data.

(b) EDA

```
skimr::skim(a3)
```

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
X1	114	0.98	12	21	0	5886	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
Y	0	1.00	FALSE	4	B: 3042, A: 2940, BA: 13, red: 5
C1	117	0.98	FALSE	4	x: 1478, w: 1472, z: 1468, y: 1465
C2	104	0.98	FALSE	4	V: 3406, U: 1645, T: 836, 44: 9

Table 1: Data summary

Name	a3
Number of rows	6000
Number of columns	7
Column type frequency:	
character	1
factor	3
numeric	3
Group variables	
	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
X2	110	0.98	-0.10	3.43	-80.00	-0.95	0.00	0.97	5.97	
X3	111	0.98	-0.83	1.06	-3.47	-1.52	-0.84	-0.15	1.90	
X4	120	0.98	0.02	1.31	-2.89	-0.67	-0.02	0.69	28.00	

(c) Clear the data**i. Data types**

As the output in Table 1, the data type of X1 is character and which should be float.
The data type of X1 could be corrected by the following code.

```
a3 <- a3 %>%
  mutate(X1 = as.numeric(X1))
```

ii. Removing numerical outliers

With the numerical part of Table 1, the P0 of X2 and the P100 of X4 shows there are outliers in these two variables, which could be filtered by the following R code,

```
a3 %>%
  filter(X2 < -20)
```

```
## # A tibble: 9 x 7
##   Y      X1    X2    X3      X4 C1    C2
##   <fct> <dbl> <dbl> <dbl>   <dbl> <fct> <fct>
## 1 A      1.64  -80 -0.834 -0.416   z     V
## 2 B      1.87  -80  0.380 -0.605   x     V
## 3 B      0.444 -80 -1.58  -0.00355 y     U
## 4 B      1.45  -80  0.478  1.45    w     V
## 5 B      1.39  -80 -0.758  0.749   z     V
## 6 A      0.0366 -80 -0.762 -0.144   z     V
## 7 A     -1.44  -80 -1.28  0.181   y     V
## 8 A      0.560 -80 -1.06  -2.31   x     V
## 9 A      0.776 -80 -0.622 -0.894   w     V
```

```
a3 %>%
  filter(X4 > 20)
```

```
## # A tibble: 5 x 7
##   Y      X1      X2      X3      X4 C1      C2
##   <fct>   <dbl>   <dbl>   <dbl> <dbl> <fct> <fct>
## 1 B      0.290  -0.666   0.784   25 w      U
## 2 A      0.995   2.23  -1.67   26 w      V
## 3 B      1.19  -1.69  -0.841   28 z      V
## 4 B      0.700   0.258  -2.41   27 x      V
## 5 A     -0.0219  0.422  -1.23   24 y      V
```

and then replace them by NA.

```
a3 <- a3 %>%
  mutate(X2 = case_when(X2 >= -20 ~ X2,
                        TRUE ~ NA)) %>%
  mutate(X4 = case_when(X4 <= 20 ~ X4,
                        TRUE ~ NA))
```

iii. Removing categorical outliers

As Table 1 shows, Y has 4 levels which should be only 2(A or B), while C2 has 4 levels which should be 3(V, U, T). The following work is to clean the unexpected levels.

```
a3 <- a3 %>%
  mutate(Y = case_when(Y %in% c("A", "B") ~ Y,
                        TRUE ~ NA)) %>%
  mutate(C2 = case_when(C2 %in% c("T", "U", "V") ~ C2,
                        TRUE ~ NA)) %>%
  mutate(Y = factor(Y, levels = c("A", "B"))) %>%
  mutate(C2 = factor(C2, levels = c("T", "U", "V")))
```

iv. Removing NA

```
a3 <- a3 %>%
  na.omit()
```

v. Final check

```
skimr::skim(a3)
```

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
Y	0	1	FALSE	2	B: 2851, A: 2763
C1	0	1	FALSE	4	x: 1409, w: 1404, z: 1403, y: 1398
C2	0	1	FALSE	3	V: 3233, U: 1582, T: 799

Table 2: Data summary

Name	a3
Number of rows	5614
Number of columns	7
Column type frequency:	
factor	3
numeric	4
Group variables	
	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
X1	0	1	0.87	1.05	-1.91	0.19	0.87	1.55	3.48	
X2	0	1	0.02	1.42	-5.03	-0.94	0.00	0.97	5.97	
X3	0	1	-0.83	1.06	-3.47	-1.52	-0.84	-0.15	1.90	
X4	0	1	0.00	1.07	-2.89	-0.67	-0.01	0.69	2.89	

All the variables are normal and correct, without any missing values, there are now 5614 observations left.

(d) Recipe

```

a3_recipe <- recipe(Y ~ ., data = a3) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_dummy(all_nominal_predictors())
a3_recipe

##

## -- Recipe -----

##

## -- Inputs

## Number of variables by role

## outcome: 1
## predictor: 6

##

## -- Operations

## * Centering and scaling for: all_numeric_predictors()

## * Dummy variables from: all_nominal_predictors()

```

(e) SVM models

```
m1 <- svm_poly(mode = "classification", degree = 3) %>%
  set_engine("kernlab")

m2 <- svm_rbf(mode = "classification") %>%
  set_engine("kernlab")
```

(f) Workflows

```
a3_wf1 <- workflow() %>%
  add_recipe(a3_recipe) %>%
  add_model(m1)

a3_wf2 <- workflow() %>%
  add_recipe(a3_recipe) %>%
  add_model(m2)
```

(g) Models Fitting

```
a3_poly <- a3_wf1 %>%
  fit(a3)
a3_poly

## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: svm_poly()
##
## -- Preprocessor -----
## 2 Recipe Steps
##
## * step_normalize()
## * step_dummy()
##
## -- Model -----
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 1
##
## Polynomial kernel function.
## Hyperparameters : degree = 3 scale = 1 offset = 1
##
## Number of Support Vectors : 755
##
## Objective Function Value : -676.3702
```

```
## Training error : 0.050053
## Probability model included.
```

```
a3_radial <- a3_wf2 %>%
  fit(a3)
a3_radial
```

```
## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: svm_rbf()
##
## -- Preprocessor -----
## 2 Recipe Steps
##
## * step_normalize()
## * step_dummy()
##
## -- Model -----
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc (classification)
## parameter : cost C = 1
##
## Gaussian Radial Basis kernel function.
## Hyperparameter : sigma = 0.079859562289001
##
## Number of Support Vectors : 1885
##
## Objective Function Value : -1467.216
## Training error : 0.060207
## Probability model included.
```

```
a3_pred <- a3 %>%
  bind_cols(
    predict(a3_poly, a3),
    predict(a3_poly, a3, type = "prob")
  ) %>%
  mutate(type = "SVM Poly") %>%
  bind_rows(
    a3 %>%
      bind_cols(
        predict(a3_radial, a3),
        predict(a3_radial, a3, type = "prob")
      ) %>%
      mutate(type = "SVM Radial")
  )
a3_pred
```

```
## # A tibble: 11,228 x 11
##   Y      X1      X2      X3      X4 C1  C2   .pred_class .pred_A .pred_B
##   <fct> <dbl> <dbl> <dbl> <dbl> <fct> <fct> <fct>      <dbl> <dbl>
## 1 B     2.10 -0.0543 -2.71  0.00119 w    V    B          7.12e-4 9.99e-1
## 2 B     1.23 -1.59  -1.37 -0.167 z    U    A          6.53e-1 3.47e-1
```

```
## 3 B      0.734 -1.44    0.0747  0.144  w    U    B      3.40e-2 9.66e-1
## 4 B      1.37  -0.467  -1.04    0.122  w    T    A      5.39e-1 4.61e-1
## 5 A      3.18   0.591  -0.219  0.346  y    U    A      1.00e+0 2.03e-5
## 6 B      1.15  -0.556  -0.847  0.0811 x    V    A      4.94e-1 5.06e-1
## 7 A      2.01  -1.98   -0.727  -1.99  w    T    A      1.00e+0 9.62e-6
## 8 A     -0.109  1.19   -1.37    0.456  y    T    A      6.11e-1 3.89e-1
## 9 B      2.44   0.0826  1.60   -1.29  y    V    B      1.25e-4 1.00e+0
## 10 A     0.767  0.587  -1.38   -0.154 z    V    B      4.82e-1 5.18e-1
## # i 11,218 more rows
## # i 1 more variable: type <chr>
```

(h) Performance metrics

i. Accuracy

```
acc <- a3_pred %>%
  group_by(type) %>%
  accuracy(
    truth = Y, estimate = .pred_class)
acc
```

```
## # A tibble: 2 x 4
##   type      .metric .estimator .estimate
##   <chr>      <chr>   <chr>      <dbl>
## 1 SVM Poly  accuracy binary      0.950
## 2 SVM Radial accuracy binary      0.940
```

ii. Sensitivity

```
sens <- a3_pred %>%
  group_by(type) %>%
  sens(truth = Y, estimate = .pred_class)
sens
```

```
## # A tibble: 2 x 4
##   type      .metric .estimator .estimate
##   <chr>      <chr>   <chr>      <dbl>
## 1 SVM Poly  sens     binary      0.949
## 2 SVM Radial sens     binary      0.942
```

iii. Specificity

```
spec <- a3_pred %>%
  group_by(type) %>%
  spec(truth = Y, estimate = .pred_class)
spec
```

```
## # A tibble: 2 x 4
##   type      .metric .estimator .estimate
##   <chr>    <chr>   <chr>      <dbl>
## 1 SVM Poly  spec    binary     0.951
## 2 SVM Radial spec    binary     0.938
```

i ROC curve and ROC-AUC

```
a3_pred %>%
  group_by(type) %>%
  roc_curve(Y,
            .pred_A) %>%
  autoplot()
```

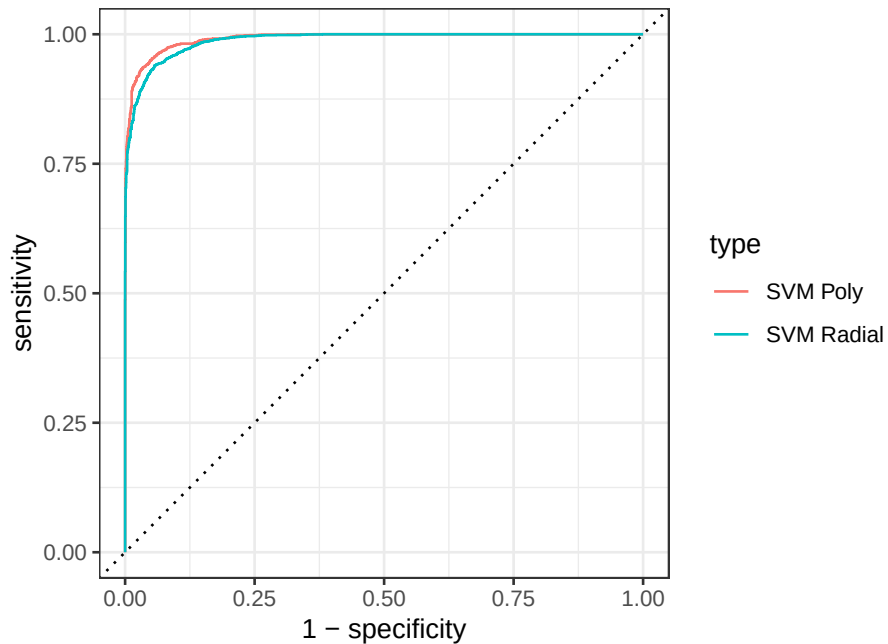


Figure 1: ROC Curves of SVM models.

The AUC of curves are

```
auc <- a3_pred %>%
  group_by(type) %>%
  roc_auc(Y, .pred_A)
auc
```

```
## # A tibble: 2 x 4
##   type      .metric .estimator .estimate
##   <chr>    <chr>   <chr>      <dbl>
## 1 SVM Poly  roc_auc binary     0.992
## 2 SVM Radial roc_auc binary     0.988
```


Table 3: Metrics Table for SVM models.

type	accuracy	sens	spec	roc_auc
SVM Poly	0.9499466	0.9493304	0.9505437	0.9915635
SVM Radial	0.9397934	0.9420919	0.9375658	0.9879856

Table 4: New data for Prediction.

Predictor	Value
X1	-0.11
X2	1.19
X3	-1.37
X4	-0.46
C1	y
C2	T

j Select the model

As Table 3 shows, the SVM model with a Polynomial kernel performance is better.

k Prediction

Use the SVM model with a Polynomial kernel to make prediction of the data in Table 4.

```
new_data <- data.frame(
  "X1" = -0.11,
  "X2" = 1.19,
  "X3" = -1.37,
  "X4" = -0.46,
  "C1" = "y",
  "C2" = "T"
)
predict(a3_poly, new_data)
```

```
## # A tibble: 1 x 1
##   .pred_class
##   <fct>
## 1 A
```

And the result is A.